SoCV Final Project

AIRTLDebug 期末專案計畫

1. 專案基本資訊

- Course：114-2 SoCV Final Project
- Selected Topic：AI for RTL Debugging
- Project Name：AIRTLDebug
- Full Name：AIRTLDebug: An LLM-Assisted Agentic Flow for RTL Bug Localization and Explanation
- Group Size：2 人
- Final Due：2026/06/20 21:00
- Submission：Team leader 的 GitHub repo
- Report：預設使用 `README.md`
- Demo Video：需要錄製，不超過 6 分鐘，並在 report 中放影片連結
- AI 使用：允許使用 AI，但需要在 final report 中揭露使用方式與第三方工具
- Main Goal：做出一個可以協助 RTL debugging 的 agentic flow，並用實際 RTL bug cases 展示它如何幫助定位 bug root cause

2. 專案定位

AIRTLDebug 是一個 lightweight LLM-assisted RTL debugging framework。

給定一個 buggy RTL benchmark、testbench 和 design specification，AIRTLDebug 會自動：

1. 讀取 RTL case
2. 執行 simulation
3. 解析 failure log
4. 擷取 suspicious signals
5. 擷取相關 RTL code blocks
6. 建立 structured LLM prompt
7. 呼叫 LLM 分析 bug root cause

8. 產生 Markdown debug report
9. 驗證 prepared fixed RTL 是否可以通過 simulation

本專案的核心想法是：

> 不讓 LLM 只靠 RTL code 猜 bug，而是用 simulation evidence 和 extracted RTL context 引導 LLM 做更有根據的 debugging。

3. 專案目標

本專案目標是設計並實作一個可以協助使用者 debug RTL design 的 command-line tool。

使用者輸入一個 buggy RTL case 後，工具會自動產生 structured debug report，內容包含：

- simulation result
- failure cycle
- failed signal
- expected value
- actual value
- suspicious signals
- suspicious RTL regions
- LLM root cause explanation
- suggested fix
- confidence score
- fix verification result

4. 題目要求對應

題目要求：

- 設計 agentic flow
- 可以與使用者互動
- 可以使用 Cursor、simulation、waveform、EDA tool 或其他 AI tool
- 展示一些 case 說明 agentic flow 確實有幫助

本專案對應方式：

題目要求	AIRTLDebug 對應
agentic flow	Case Loader、Simulation Runner、Log Parser、RTL Context Extractor、LLM Agent、Report Generator、Fix Verification
interacts with users	支援 CLI mode 與 interactive mode
leverage simulation / EDA tool	使用 Icarus Verilog 執行 RTL simulation
leverage AI tools	使用 Gemini API / OpenAI API 產生 root cause analysis
demonstrate cases	使用 counter、FIFO、FSM 三個 RTL bug cases
help users debug	產生 suspicious signal、RTL region、root cause、fix suggestion、verification result

5. 最終 Demo 目標

使用者執行基本 debug flow

```
python src/rtl_debug_agent.py \
  --case benchmarks/counter_off_by_one \
  --model gemini \
  --output outputs/counter_off_by_one/debug_report.md \
  --fix-check
```

工具預期輸出摘要

```
Bug Case: counter_off_by_one
Buggy Simulation: FAILED
Failure Cycle: 17
Failed Signal: done
Expected: 0
Actual: 1

Suspicious Signals:
- count
- done
- enable

Suspicious RTL Region:
- design_buggy.sv: always_ff / sequential always block around count and done update

Root Cause:
done is asserted one cycle earlier than expected because the RTL compares the current count value instead of the next count value.

Suggested Fix:
Compute next_count first and compare next_count with MAX.

Fix Verification:
Prepared fixed RTL simulation PASS.
```

6. 重要限制說明

本專案的 fix verification 不是自動修改 RTL source code。

正確描述：

> AIRTLDebug reports a suggested fix and verifies a prepared fixed RTL version to check whether the intended correction resolves the failure.

避免寫成：

> AIRTLDebug automatically patches RTL and verifies the generated patch.

除非後續真的加入 automatic patch generation，否則 final report 要明確說明：

- LLM 會產生 suggested fix
- tool 會驗證 case folder 中事先準備的 `design_fixed.sv`
- verification 目標是確認 ground-truth fix 可以讓 simulation PASS

7. 系統架構

```
User
  |
  v
CLI / Interactive Mode
  |
  v
Case Loader
  |
  v
Simulation Runner
  |
  v
Failure Log Parser
  |
  v
RTL Context Extractor
  |
  v
Prompt Builder
  |
  v
LLM Debug Agent
  |
  v
Structured Debug Report
  |
```

```
  v
Fix Verification
```

8. Agentic Flow 說明

AIRTLDebug 將 RTL debugging 拆成多個 tool-using steps。

Step 1：Case Loading

讀取 benchmark case 中的檔案：

- `design_buggy.sv`
- `design_fixed.sv`
- `tb.sv`
- `spec.md`
- `expected_root_cause.md`
- `ground_truth.json`

Step 2：Simulation

使用 Icarus Verilog 執行 buggy RTL simulation。

```
iverilog -g2012 -o outputs/<case_name>/sim_buggy.out \
  benchmarks/<case_name>/design_buggy.sv \
  benchmarks/<case_name>/tb.sv

vvp outputs/<case_name>/sim_buggy.out
```

Step 3：Failure Log Parsing

從 simulation output 擷取：

- status
- failure cycle
- failed signal
- expected value
- actual value
- assertion message
- mismatch message

Step 4：RTL Context Extraction

根據 failed signal 和 failure log，從 RTL code 中擷取：

- suspicious signal list
- related line numbers
- related always_ff / sequential always block
- related always_comb / combinational always @(*) block
- related assign statement
- state transition block if available

Step 5：Prompt Building

將以下資訊整理成 structured prompt：

- design specification
- buggy RTL

- testbench summary
- parsed failure log
- suspicious signals
- suspicious RTL blocks
- ground rule：不要編造不存在的 signal
- ground rule：回答必須引用 simulation evidence

Step 6：LLM Debugging

LLM 輸出固定格式：

- bug summary
- suspicious signals
- suspicious RTL lines or regions
- root cause explanation
- suggested fix
- evidence from simulation log
- confidence score

Step 7：Report Generation

產生 Markdown debug report：

```
outputs/<case_name>/debug_report.md
```

Step 8：Fix Verification

如果使用者加上 `--fix-check`，tool 會執行 `design_fixed.sv` 並檢查 simulation 是否 PASS。

9. CLI 功能設計

9.1 基本指令

```
python src/rtl_debug_agent.py --case benchmarks/counter_off_by_one
```

9.2 指定模型

```
python src/rtl_debug_agent.py \
--case benchmarks/counter_off_by_one \
--model gemini
```

9.3 指定輸出 report

```
python src/rtl_debug_agent.py \
--case benchmarks/counter_off_by_one \
--output outputs/counter_off_by_one/debug_report.md
```

9.4 跑 fixed RTL verification

```
python src/rtl_debug_agent.py \
--case benchmarks/counter_off_by_one \
--fix-check
```

9.5 不呼叫 LLM，只跑 simulation、parser、extractor

```
python src/rtl_debug_agent.py \
--case benchmarks/counter_off_by_one \
--no-llm
```

9.6 Interactive mode

```
python src/rtl_debug_agent.py \
--case benchmarks/fifo_full_flag_bug \
--interactive
```

9.7 跑全部 cases

```
bash scripts/run_all_cases.sh
```

9.8 跑 experiment comparison

```
python scripts/run_experiments.py
```

10. Interactive Mode 設計

Interactive mode 的目的不是做 GUI，而是讓使用者可以 step-by-step 檢查 debug evidence。

互動流程範例

```
AIRTLDebug found a simulation failure.

Case: fifo_full_flag_bug
Failure cycle: 12
Failed signal: full
Expected: 1
Actual: 0

Suspicious signals:
- full
- empty
- wr_ptr
- rd_ptr
- wrap_bit

Choose next action:
1. Show failure log
2. Show suspicious RTL block
3. Ask LLM for root cause
4. Ask LLM to explain signal relation
5. Run fixed RTL verification
6. Export debug report
0. Exit
```

Interactive mode 的價值

- 符合題目中 interacts with users 的要求
- demo 時更容易展示 tool 的 debugging process
- 可以讓使用者先看 evidence，再看 LLM 分析
- 不需要額外 GUI，實作成本低

11. Module Design

11.1 Case Loader

File

```
src/case_loader.py
```

功能

負責讀取 benchmark case。

每個 case folder 包含：

```
design_buggy.sv
design_fixed.sv
tb.sv
spec.md
expected_root_cause.md
ground_truth.json
```

輸出資料

```
{
  "case_name":"counter_off_by_one",
  "buggy_rtl":"...",
  "fixed_rtl":"...",
  "testbench":"...",
  "spec":"...",
  "expected_root_cause":"...",
  "ground_truth": {
    "suspicious_signals": ["count","done"],
    "suspicious_regions": [
      {
        "file":"design_buggy.sv",
        "start_line":23,
        "end_line":35
      }
    ]
  }
}
```

11.2 Simulation Runner

File

```
src/sim_runner.py
```

功能

負責執行 RTL simulation。

Compile command

```
iverilog -g2012 -o outputs/<case_name>/sim_buggy.out \
  benchmarks/<case_name>/design_buggy.sv \
  benchmarks/<case_name>/tb.sv
```

Run command

```
vvp outputs/<case_name>/sim_buggy.out
```

回傳 status

- `PASS`
- `FAIL`
- `COMPILE_ERROR`

- `RUNTIME_ERROR`

Log output

```
outputs/<case_name>/buggy_sim.log
outputs/<case_name>/fixed_sim.log
```

11.3 Failure Log Parser

File

```
src/log_parser.py
```

功能

從 simulation log 擷取可機器讀取的 failure information。

Testbench failure message 格式

所有 benchmark 的 testbench 都應該使用固定格式輸出 failure message。

```
[FAIL] cycle=17 signal=done expected=0 actual=1 message="done asserted too early"
```

Parsed JSON

```
{
  "status":"FAILED",
  "failure_cycle":17,
  "failed_signal":"done",
  "expected":"0",
  "actual":"1",
  "message":"done asserted too early"
}
```

Parser fallback

如果 parser 找不到資訊：

```
{
  "status":"FAILED",
  "failure_cycle":null,
  "failed_signal":null,
  "expected":null,
  "actual":null,
  "message":"Simulation failed, but no structured failure message was found."
}
```

11.4 RTL Code Extractor

File

```
src/rtl_extractor.py
```

功能

根據 failure log 中出現的 signal，擷取 RTL 中相關 code region。

簡化策略

不做完整 SystemVerilog parser，採用 text-based extraction：

1. 找出 failed signal 出現的 line

2. 向上和向下擷取附近 code
3. 優先擷取完整：
    - always_ff / sequential always block
    - always_comb / combinational always @(*) block
    - assign statement
    - case block
    - if block
4. 建立 suspicious signal list
5. 建立 suspicious code block list

輸出格式

```
{
  "suspicious_signals": ["count","done","enable"],
  "suspicious_code_blocks": [
    {
      "type": "sequential_always",
      "file":"design_buggy.sv",
      "start_line":23,
      "end_line":35,
      "code":"always_ff @(posedge clk) begin ..."
    }
  ]
}
```

11.5 Prompt Builder

File

```
src/prompt_builder.py
```

功能

根據不同 experiment setting 建立 prompt。

支援三種 prompt mode：

- `llm_only`
- `llm_with_log`
- `full_flow`

Prompt input

Mode	Input
LLM only	RTL code + spec
LLM + log	RTL code + spec + simulation log
Full flow	spec + parsed log + suspicious signals + suspicious RTL blocks

11.6 LLM Client

File

```
src/llm_client.py
```

功能

負責呼叫 LLM API。

支援 backend

- Gemini API
- OpenAI API
- mock mode for no-key testing

API key

使用 `.env` 讀取，不要把 API key push 到 repo。

`.env.example`

```
GEMINI_API_KEY=
OPENAI_API_KEY=
DEFAULT_MODEL=gemini
```

Error handling

需要處理：

- API key missing
- rate limit
- timeout
- invalid response
- network error

如果 LLM API 失敗，report 中要清楚顯示：

```
LLM analysis failed because API key is missing.
```

11.7 Report Writer

File

```
src/report_writer.py
```

功能

產生 Markdown debug report。

Report format

```
# Debug Report: <case_name>

## Simulation Result
## Failure Evidence
## Extracted RTL Context
## LLM Root Cause Analysis
## Suggested Fix
## Fix Verification
## Comparison with Ground Truth
```

11.8 Evaluator

File

```
src/evaluator.py
```

功能

比較 LLM output 和 ground truth。

評估項目

- root cause correctness
- suspicious signal accuracy
- suspicious region accuracy
- evidence consistency
- fix verification result

評分標籤

- `Correct`
- `Partial`
- `Wrong`
- `N/A`

12. Benchmark Cases

12.1 Case 1：Counter Off-by-One Bug

Case name

```
counter_off_by_one
```

Bug type

Counter termination condition bug。

Correct behavior

`done` should be asserted when `count == MAX`。

Buggy behavior

`done` is asserted when `count == MAX - 1`。

Suspicious signals

- `count`
- `done`
- `enable`

Expected root cause

`done` is asserted one cycle earlier than expected because the RTL uses the wrong comparison condition.

Difficulty

Low。

Purpose

作為第一個完整 demo case，確保 pipeline 可以穩定跑通。

12.2 Case 2：FIFO Full Flag Bug

Case name

```
fifo_full_flag_bug
```

Bug type

FIFO pointer / flag logic bug。

Correct behavior

`full` should consider write pointer, read pointer, and wrap-around bit。

Buggy behavior

`full` only compares `wr_ptr == rd_ptr`，造成 full 和 empty ambiguity。

Suspicious signals

- `wr_ptr`
- `rd_ptr`
- `full`
- `empty`
- `wrap_bit`

Expected root cause

The FIFO cannot distinguish full from empty because the full flag logic ignores wrap-around information.

Difficulty

Medium。

Purpose

展示 AIRTLDebug 可以處理 data-structure-like RTL bug。

12.3 Case 3：Vending Machine FSM Bug

Case name

```
vending_machine_fsm_bug
```

Bug type

FSM transition or output timing bug。

Correct behavior

When accumulated coin value reaches price, `dispense` should be asserted at the correct state or cycle。

Buggy behavior

FSM misses a transition or asserts `dispense` at the wrong state。

Suspicious signals

- `state`
- `next_state`
- `coin`
- `credit`
- `dispense`
- `refund`

Expected root cause

The FSM transition or output logic uses the wrong state condition, causing `dispense` to be missed or asserted at the wrong cycle.

Difficulty

Medium。

Purpose

展示 AIRTLDebug 可以處理 controller / FSM debugging。

13. Benchmark Folder Format

每個 benchmark case 應該包含：

```
benchmarks/<case_name>/
  design_buggy.sv
  design_fixed.sv
  tb.sv
  spec.md
  expected_root_cause.md
  ground_truth.json
```

ground_truth.json format

```
{
  "case_name":"counter_off_by_one",
  "bug_type":"counter off-by-one",
  "root_cause":"done is asserted one cycle too early because the comparison condition uses MAX - 1.",
  "suspicious_signals": ["count","done"],
  "suspicious_regions": [
    {
      "file":"design_buggy.sv",
      "start_line":23,
      "end_line":35,
      "description":"always_ff / sequential always block that updates count and done"
    }
  ],
  "expected_fix":"Compare count with MAX, or compute next_count and compare next_count with MAX."
}
```

14. Repo Structure

```
AIRTLDebug/
  README.md
  eval.txt
  requirements.txt
  .env.example
  .gitignore

  src/
    rtl_debug_agent.py
    case_loader.py
    sim_runner.py
    log_parser.py
    rtl_extractor.py
    prompt_builder.py
    llm_client.py
    report_writer.py
    evaluator.py

  prompts/
    debug_prompt.md
    debug_prompt_llm_only.md
    debug_prompt_with_log.md
    debug_prompt_full_flow.md
    fix_prompt.md

  benchmarks/
    counter_off_by_one/
      design_buggy.sv
```

```
      design_fixed.sv
      tb.sv
      spec.md
      expected_root_cause.md
      ground_truth.json

    fifo_full_flag_bug/
      design_buggy.sv
      design_fixed.sv
      tb.sv
      spec.md
      expected_root_cause.md
      ground_truth.json

    vending_machine_fsm_bug/
      design_buggy.sv
      design_fixed.sv
      tb.sv
      spec.md
      expected_root_cause.md
      ground_truth.json

  scripts/
    run_all_cases.sh
    run_experiments.py

  outputs/
    .gitkeep

  docs/
  architecture.md
  experiment_results.md
  demo_script.md
  sample_reports/
    counter_off_by_one_debug_report.md
```

15. Prompt Design

15.1 Full Flow Debug Prompt

```
You are an RTL debugging assistant.

You are given:
1. Design specification
2. Parsed simulation failure
3. Suspicious signals
4. Extracted RTL code blocks
5. Optional testbench context

Task:
Find the most likely root cause of the RTL bug.

Output format:
1. Bug summary
2. Suspicious signals
3. Suspicious RTL lines or regions
4. Root cause explanation
5. Suggested fix
6. Evidence from simulation log
7. Confidence score

Rules:
- Base your answer only on the given RTL code and simulation evidence.
- Do not invent signals that do not exist.
- If the evidence is insufficient, state what is missing.
- Explain why the failure happens.
- Explain how the suggested fix addresses the failure.
```

15.2 LLM Only Prompt

```
You are an RTL code review assistant.

You are given:
1. Design specification
2. Full buggy RTL code
```

```
Task:
Find possible bugs in the RTL code.

Output format:
1. Possible bug summary
2. Suspicious signals
3. Suspicious RTL regions
4. Explanation
5. Confidence score

Rules:
- Do not use simulation evidence because it is not provided.
- Mark uncertainty clearly.
- Do not invent signals that do not exist.
```

15.3 LLM + Log Prompt

```
You are an RTL debugging assistant.

You are given:
1. Design specification
2. Full buggy RTL code
3. Simulation failure log

Task:
Find the most likely root cause of the failure.

Output format:
1. Bug summary
2. Suspicious signals
3. Suspicious RTL regions
4. Root cause explanation
5. Evidence from simulation log
6. Confidence score

Rules:
- Use the simulation failure log as evidence.
- Do not invent signals that do not exist.
- If the failure log is insufficient, state what information is missing.
```

16. Debug Report Format

每個 case 產生一份 Markdown report。

Output path

```
outputs/<case_name>/debug_report.md
```

Report template

```
# Debug Report: <case_name>

## 1. Case Information

- Case name:
- Bug type:
- Model:
- Prompt mode:

## 2. Simulation Result

- Buggy RTL simulation:
- Failure cycle:
- Failed signal:
- Expected:
- Actual:
- Failure message:

## 3. Failure Evidence

<original parsed failure evidence>

## 4. Extracted RTL Context
```

```
### Suspicious Signals

- signal 1
- signal 2

### Suspicious RTL Blocks

- file:
- start line:
- end line:
- code block:

## 5. LLM Root Cause Analysis

- Bug summary:
- Root cause:
- Evidence:
- Confidence score:

## 6. Suggested Fix

<LLM suggested fix>

## 7. Fix Verification

- Fixed RTL simulation:
- Result:

## 8. Comparison with Ground Truth

- Root cause correctness:
- Suspicious signal correctness:
- Suspicious region correctness:
- Evidence consistency:

## 9. Notes

- Limitations:
- Missing evidence if any:
```

17. Experimental Design

17.1 Experiment 1：LLM Only vs LLM + Simulation Log vs Full Flow

Goal

比較不同 input setting 對 LLM debugging 的影響。

Settings

Setting	Input to LLM	Purpose
LLM only	RTL code + spec	測試 LLM 只靠 code review 能否找 bug
LLM + log	RTL code + spec + simulation log	測試 simulation evidence 是否有幫助
Full flow	spec + parsed log + suspicious signals + suspicious RTL blocks	測試 AIRTLDebug 的 context extraction 是否讓分析更聚焦

Evaluation metrics

Metric	Definition
Root cause correctness	是否指出真正 bug 原因
Signal accuracy	是否包含 ground-truth suspicious signals
Region accuracy	是否定位到正確 always block / assign / case block
Evidence consistency	explanation 是否和 simulation log 一致
Fix usefulness	suggested fix 是否能對應 ground truth fix
Fixed pass	prepared fixed RTL 是否 simulation PASS

17.2 Experiment 2：Different Bug Types

Goal

觀察 AIRTLDebug 對不同 RTL bug 類型的效果。

Cases

Case	Bug Type	Difficulty
counter_off_by_one	Counter condition bug	Low
fifo_full_flag_bug	FIFO flag / pointer bug	Medium
vending_machine_fsm_bug	FSM transition / output bug	Medium

Questions

- 哪種 bug 最容易被定位
- 哪種 bug 最需要 simulation log
- 哪種 bug 最容易讓 LLM 誤判
- extracted RTL blocks 是否比 full RTL 更集中
- suggested fix 是否和 ground truth 一致

17.3 Experiment 3：Full RTL vs Extracted RTL Blocks

Goal

比較 full RTL input 和 extracted context input 的差異。

Settings

Setting	Description
Full RTL	將完整 RTL code 給 LLM
Extracted blocks	只給 suspicious signals 和相關 RTL blocks

Evaluation

- root cause correctness
- suspicious region accuracy
- report length
- hallucination frequency
- explanation clarity

18. Experimental Result Table Format

Final README 中不要先填預設結果。要在實際跑完後填入。

Evaluation protocol table

Metric	Correct	Partial	Wrong
Root cause	完整指出 ground-truth root cause	只指出部分原因	指錯原因
Signal	包含主要 suspicious signals	只找到部分 signals	找錯 signals
Region	定位到正確 block	定位到附近 block	定位錯誤 block
Evidence	explanation 和 log 一致	部分一致	和 log 矛盾
Fix	fix 對應 ground truth	fix 方向部分正確	fix 無效

Final result table

Case	LLM Only	LLM + Log	Full Flow	Root Cause	Signal	Region
counter_off_by_one	TBD	TBD	TBD	TBD	TBD	TBD
fifo_full_flag_bug	TBD	TBD	TBD	TBD	TBD	TBD

Case	LLM Only	LLM + Log	Full Flow	Root Cause	Signal	Region
vending_machine_fsm_bug	TBD	TBD	TBD	TBD	TBD	TBD

Final summary table

Case	Bug Type	Best Setting	Main Finding
counter_off_by_one	Counter	TBD	TBD
fifo_full_flag_bug	FIFO	TBD	TBD
vending_machine_fsm_bug	FSM	TBD	TBD

19. Key Contributions

Final README 可以寫以下 contribution。

Contribution 1：Simulation-guided RTL debugging flow

AIRTLDebug uses simulation failure evidence to guide LLM-based RTL debugging.

Contribution 2：Structured failure parsing

The tool converts raw simulation output into structured JSON containing failure cycle, failed signal, expected value, actual value, and assertion message.

Contribution 3：RTL context extraction

The tool extracts suspicious RTL code blocks based on failed signals, reducing irrelevant context before querying the LLM.

Contribution 4：Structured LLM debug report

The LLM output is converted into a reproducible Markdown report with root cause, evidence, suspicious signals, suggested fix, and confidence score.

Contribution 5：Ground-truth-based evaluation

Each benchmark contains ground truth root cause, suspicious signals, and suspicious RTL regions, allowing comparison between LLM-only debugging and the full AIRTLDebug flow.

20. Third-Party Tools and Prior Work Disclosure

README 必須揭露哪些東西不是本專案貢獻。

Third-party tools

```
Third-party tools:
- Icarus Verilog: used as the RTL simulation backend.
- Gemini API / OpenAI API: used as the LLM backend.
- Python standard library: used for CLI, file I/O, subprocess execution, and JSON processing.
- python-dotenv: used to load API keys from .env if used.
- Cursor / ChatGPT: used as AI coding and writing assistants if applicable.
```

Not counted as our contribution

```
Not counted as our contribution:
- The LLM model itself.
- The Verilog simulator.
- Any third-party Python package.
- Any external benchmark or reference RTL code if used.
- Any code written before this semester.
```

Our contribution

```
Our contribution:
- The AIRTLDebug agentic debugging flow.
```

```
- The benchmark cases created for this final project.
- The simulation runner integration.
- The failure log parser.
- The RTL context extractor.
- The prompt templates.
- The debug report generation flow.
- The experiment design and evaluation.
```

21. README.md 必放內容

README 預設是 final project report，因此需要包含：

```
# AIRTLDebug

## Group Members
## Project Overview
## Motivation
## Key Contributions
## System Architecture
## Algorithm
## Implementation Details
## Installation
## How to Run
## How to Test
## Benchmark Cases
## Experimental Setup
## Experimental Results
## Demo Video
## Third-Party Tools and Prior Work Disclosure
## Limitations
## References
```

22. Algorithm Pseudo Code

```
Algorithm: AIRTLDebug

Input:
  RTL bug case directory

Output:
  Structured RTL debug report

Steps:
  1. Load design_buggy.sv, design_fixed.sv, tb.sv, spec.md, expected_root_cause.md, and ground_truth.json.
  2. Run buggy RTL simulation using Icarus Verilog.
  3. Save raw simulation output as buggy_sim.log.
  4. Parse simulation log to extract status, failure cycle, failed signal, expected value, actual value, and message.
  5. Search RTL code for signals mentioned in the failure log.
  6. Extract related sequential always blocks, combinational always blocks, assign statements, and case blocks.
  7. Build an LLM prompt using the spec, parsed failure log, suspicious signals, and extracted RTL blocks.
  8. Ask the LLM to identify suspicious signals, suspicious RTL region, root cause, suggested fix, evidence, and confidence score.
  9. Write the LLM result and extracted evidence into a Markdown debug report.
  10. If fix-check is enabled, run fixed RTL simulation and include the result in the report.
  11. Compare the LLM output with ground_truth.json for experiment evaluation.
```

23. Implementation Details

23.1 Simulation output design

每個 testbench 要產生固定格式的 failure message：

```
[FAIL] cycle=<cycle> signal=<signal> expected=<expected> actual=<actual> message="<message>"
```

PASS 時輸出：

```
[PASS] all checks passed
```

這樣 `log_parser.py` 可以穩定解析，不需要做複雜 NLP。

23.2 Output directory design

每個 case 的中間結果放在：

```
outputs/<case_name>/
```

範例：

```
outputs/counter_off_by_one/
  buggy_sim.log
  fixed_sim.log
  parsed_failure.json
  extracted_context.json
  llm_response_raw.md
  debug_report.md
```

23.3 Mock mode

為了避免 demo 時 API key 或 rate limit 問題，建議加入 mock mode：

```
python src/rtl_debug_agent.py \
--case benchmarks/counter_off_by_one \
--model mock
```

Mock mode 可以讀取：

```
benchmarks/<case_name>/expected_root_cause.md
```

並產生固定格式 report。

注意：final report 要誠實說明 mock mode 只是 backup，不算主要 LLM result。

23.4 API key safety

`.env` 不可以 push 到 repo。

`.gitignore` 要包含：

```
.env
*.out
*.vcd
*.o
__pycache__/
outputs/*/
!outputs/.gitkeep
```

23.5 Sample report submission policy

The `outputs/` directory is used for generated runtime results, including simulation logs, parsed JSON files, raw LLM responses, and generated debug reports. These files are ignored by `.gitignore` to avoid submitting unnecessary logs.

For the final submission, selected cleaned debug reports will be copied to `docs/sample_reports/` so that the instructor can inspect representative AIRTLDebug outputs without keeping all generated logs in the repository.

23.6 Icarus Verilog compatibility

AIRTLDebug uses Icarus Verilog as the simulation backend. To keep the benchmark cases easy to compile and reproduce, the RTL code will avoid advanced SystemVerilog features if they cause compatibility issues with Icarus Verilog.

If `always_ff` or `always_comb` causes compatibility issues in a specific environment, the benchmark RTL can be rewritten using standard `always` blocks while preserving the same RTL behavior. The goal of the benchmark is to demonstrate the debugging flow, not to depend on simulator-specific SystemVerilog features.

24. 五週時程規劃

Week 1：Repo skeleton + counter case + basic simulation

Goal

完成基本 repo 架構，讓第一個 counter case 可以跑 simulation。

A tasks

- 建立 `counter_off_by_one`
- 撰寫 `design_buggy.sv`
- 撰寫 `design_fixed.sv`
- 撰寫 `tb.sv`
- 撰寫 `spec.md`
- 撰寫 `expected_root_cause.md`
- 撰寫 `ground_truth.json`
- 確認 buggy version FAIL
- 確認 fixed version PASS
- 實作 basic `sim_runner.py`

B tasks

- 建立 GitHub repo
- 建立資料夾架構
- 建立 `.gitignore`
- 建立 `requirements.txt`
- 建立 `.env.example`
- 建立 README skeleton
- 設計第一版 prompt template

Week 1 done criteria

```
python src/rtl_debug_agent.py --case benchmarks/counter_off_by_one --no-llm
```

至少可以輸出：

```
Simulation failed.
Failure log saved.
```

Week 2：完成三個 benchmark + parser + extractor

Goal

完成所有 benchmark cases，並能產生 parsed JSON 和 extracted context。

A tasks

- 完成 `fifo_full_flag_bug`
- 完成 `vending_machine_fsm_bug`
- 每個 case 都有 fixed version
- 每個 case 都有 testbench
- 每個 case 都有 structured failure message
- 實作 `log_parser.py`

- 實作 `rtl_extractor.py`
- 確認三個 cases 都能產生 parsed JSON

B tasks

- 根據 A 的 intermediate JSON 設計 prompt input format
- 撰寫 `prompt_builder.py`
- 協助確認 extracted context 是否適合 LLM 使用
- 開始設計 report format

Week 2 done criteria

每個 case 都能產生：

```
outputs/<case_name>/buggy_sim.log
outputs/<case_name>/parsed_failure.json
outputs/<case_name>/extracted_context.json
```

Week 3：整合 LLM Agent + report generation

Goal

讓工具可以自動產生 debug report。

A tasks

- 實作 CLI main flow
- 支援 `--case`
- 支援 `--output`
- 支援 `--fix-check`
- 支援 `--no-llm`
- 協助檢查 LLM output 是否符合 RTL 事實
- 補強 benchmark 的 log readability

B tasks

- 實作 `llm_client.py`
- 支援 Gemini API 或 OpenAI API
- 支援 `.env`
- 支援 mock mode
- 實作 `report_writer.py`
- 完成 full flow prompt
- 產生第一版 debug report

Week 3 done criteria

三個 case 都能自動產生：

```
outputs/<case_name>/debug_report.md
```

每份 report 至少包含：

```
Bug summary
Suspicious signals
Suspicious RTL region
Root cause
```

```
Suggested fix
Evidence from simulation log
Fix verification result
```

Week 4：Interactive mode + experiments

Goal

完成 interactive mode 與實驗比較。

A tasks

- 加入 `--interactive`
- 支援使用者查看 failure log
- 支援使用者查看 suspicious RTL block
- 支援使用者執行 fix verification
- 協助 B 跑三個 benchmark 的 raw results

B tasks

- 實作 `scripts/run_experiments.py`
- 跑三種設定：
  - LLM only
  - LLM + log
  - Full flow
- 整理 result table
- 撰寫 `docs/experiment_results.md`
- 撰寫 `src/evaluator.py`
- 繪製 architecture flow chart

Week 4 done criteria

README 中有：

```
Experimental Setup
Evaluation Metrics
Experiment Results
Discussion
```

且 result table 使用實際實驗結果，不預填假資料。

Week 5：Final report + demo video + repo cleanup

Goal

完成可提交版本。

A tasks

- 檢查所有 benchmark 可重現
- 確認所有 fixed version PASS
- 補上 RTL / testbench 註解
- 檢查 `.gitignore`
- 確認沒有 push `.o`、binary、core dump、不必要 logs
- 協助確認 README 的 technical details 正確

B tasks

- 完成 README.md
- 完成 demo video script
- 錄製 6 分鐘內 demo video
- 上傳 demo video
- 在 README 放影片連結
- 完成 `eval.txt`
- 最後檢查 repo
- 如果 repo private，加入 instructor `ric2k1`

Week 5 done criteria

Repo 包含：

```
README.md
eval.txt
requirements.txt
.env.example
.gitignore
src/
benchmarks/
prompts/
scripts/
docs/
outputs/.gitkeep
```

README 包含：

```
Group members
How to compile, run, and test
Key contributions
Prior work and third-party tools
Algorithm flow chart or pseudo code
Algorithm description
Implementation details
Experimental results
References
Demo video link
```

25. 兩人分工

分工原則

本專案採用 50 / 50 分工。

- A：負責 core implementation 與 RTL benchmark，讓工具可以實際執行
- B：負責 LLM API、prompt、report、experiment、demo video，讓專案成為完整的 AI-assisted debugging flow

雖然 A 會寫比較多核心程式，但 B 負責 AI 整合與期末交付內容，因此兩邊工作量可以公平算 50 / 50。

A：Core Implementation and RTL Benchmark Engineer

A-1. RTL Bug Benchmark

A 負責建立所有 RTL debugging cases。

細項

- 設計至少 3 個 RTL bug cases
  - `counter_off_by_one`
  - `fifo_full_flag_bug`

- `vending_machine_fsm_bug`

- 每個 case 建立：
  - `design_buggy.sv`
  - `design_fixed.sv`
  - `tb.sv`
  - `spec.md`
  - `expected_root_cause.md`
  - `ground_truth.json`
- 確認 buggy version 會 simulation FAIL
- 確認 fixed version 會 simulation PASS
- 設計 testbench error message，讓 failure log 清楚可解析
- 標記每個 case 的 correct suspicious signal
- 標記每個 case 的 correct suspicious RTL region
- 建立 ground truth root cause

A 產出

```
benchmarks/
  counter_off_by_one/
  fifo_full_flag_bug/
  vending_machine_fsm_bug/
```

A-2. Simulation Runner

A 負責讓工具可以自動跑 RTL simulation。

細項

- 使用 `iverilog` / `vvp` 執行 simulation
- 實作 compile command
- 實作 run command
- 支援指定 case path
- 支援 buggy RTL simulation
- 支援 fixed RTL simulation
- 將 simulation output 存成 log
- 回傳 simulation status：
  - `PASS`
  - `FAIL`
  - `COMPILE_ERROR`
  - `RUNTIME_ERROR`

A 產出

```
src/sim_runner.py
```

A-3. Failure Log Parser

A 負責解析 simulation failure log。

細項

- 從 log 中擷取：
  - failure cycle
  - failed signal
  - expected value
  - actual value
  - assertion message
  - mismatch message
- 將 parser result 存成 JSON
- 設計固定 JSON schema 給 B 使用
- 處理 parser 找不到資訊時的 fallback
- 確認三個 benchmark 的 log 都能被 parser 正確解析

A 產出

```
src/log_parser.py
```

A-4. RTL Code Extractor

A 負責從 RTL 中擷取可疑 code block。

細項

- 根據 failed signal 搜尋 RTL code
- 找出 signal 出現的 line number
- 擷取附近 code region
- 擷取相關：
  - always_ff / sequential always block
  - always_comb / combinational always @(*) block
  - assign statement
  - case block
  - state transition block
- 建立 suspicious signal list
- 建立 suspicious code block list
- 輸出給 B 作為 LLM prompt input

A 產出

```
src/rtl_extractor.py
```

A-5. CLI Main Flow

A 負責主程式骨架與 basic pipeline。

細項

- 實作 CLI argument parsing
- 支援：
  - `--case`
  - `--output`

    - `--fix-check`
    - `--no-llm`
    - `--interactive`
- 串接：
    - case loader
    - simulation runner
    - log parser
    - RTL Context extractor
- 產生中間 JSON 給 B
- 支援單一 case 執行
- 支援 batch run

A 產出

```
src/rtl_debug_agent.py
src/case_loader.py
scripts/run_all_cases.sh
```

A-6. Fix Verification

A 負責驗證 prepared fixed RTL 是否真的修好。

細項

- 支援 `--fix-check`
- 自動改跑 `design_fixed.sv`
- 檢查 fixed version simulation result
- 將 fixed result 寫進 JSON
- 提供 B 放進 final debug report

A 產出

```
src/sim_runner.py
src/rtl_debug_agent.py
```

A-7. 提供給 B 的 intermediate JSON

```
{
  "case_name":"counter_off_by_one",
  "spec":"...",
  "buggy_rtl":"...",
  "testbench":"...",
  "simulation_log":"...",
  "parsed_failure": {
    "status":"FAILED",
    "cycle":17,
    "signal":"done",
    "expected":"0",
    "actual":"1"
  },
  "suspicious_signals": ["count","done"],
  "suspicious_code_blocks": [
    {
      "type": "sequential_always",
      "file":"design_buggy.sv",
      "start_line":23,
      "end_line":35,
      "code":"..."
    }
  ],
```

```
  "fix_verification": {
    "status":"PASS"
  }
}
```

B：LLM Integration, Report, and Demo Engineer

B-1. LLM API Integration

B 負責串接 LLM API。

細項

- 選定 LLM backend：
  - Gemini API
  - OpenAI API
  - 或其他可用模型
- 實作 API client
- 支援從 `.env` 讀 API key
- 支援 mock mode
- 設計錯誤處理：
  - API key missing
  - rate limit
  - timeout
  - invalid response
- 將 A 產生的 JSON 轉成 LLM input
- 儲存 raw LLM response

B 產出

```
src/llm_client.py
.env.example
```

B-2. Prompt Design

B 負責設計 prompt。

細項

- 設計 debug prompt template
- 設計 fix suggestion prompt template
- 設計 LLM output format
- 要求 LLM 根據 evidence 回答
- 要求 LLM 不要編造不存在的 signal
- 根據不同 experiment 調整 prompt：
  - LLM only
  - LLM + simulation log
  - full agentic flow

B 產出

```
prompts/debug_prompt.md
prompts/debug_prompt_llm_only.md
```

```
prompts/debug_prompt_with_log.md
prompts/debug_prompt_full_flow.md
prompts/fix_prompt.md
```

B-3. Debug Report Generator

B 負責產生 Markdown debug report。

細項

- 將 LLM response 轉成 Markdown report
- Report 包含：
  - case name
  - simulation status
  - failure evidence
  - suspicious signals
  - suspicious RTL region
  - root cause explanation
  - suggested fix
  - confidence score
  - fix verification result
  - ground truth comparison
- 每個 case 產生一份 report
- 支援輸出到 `outputs/`

B 產出

```
src/report_writer.py
outputs/<case_name>/debug_report.md
```

B-4. Experiment Comparison

B 負責設計與整理實驗比較。

細項

- 設計三種設定：
  - LLM only
  - LLM + simulation log
  - Our full flow
- 對每個 case 跑三種設定
- 整理比較表
- 判斷結果：
  - `Correct`
  - `Partial`
  - `Wrong`
- 整理 root cause 是否正確
- 整理 suspicious signal 是否正確
- 整理 suspicious RTL region 是否正確
- 整理 fixed version 是否 PASS

B 產出

```
docs/experiment_results.md
scripts/run_experiments.py
src/evaluator.py
```

B-5. README Final Report

B 負責整合 README，A 提供 technical details。

README 至少包含

- group members
- project overview
- motivation
- how to compile
- how to run
- how to test
- key research contributions
- third-party tools disclosure
- algorithm flow chart
- pseudo code
- implementation details
- benchmark cases
- experimental results
- limitations
- references
- demo video link

B 產出

```
README.md
```

B-6. Demo Video

B 負責 demo video。

細項

- 撰寫 demo script
- 規劃 6 分鐘內影片流程
- 錄製工具執行畫面
- 展示一個完整 case：
  - run command
  - simulation fail
  - parsed failure
  - suspicious RTL block
  - LLM report
  - root cause
  - fix verification pass

- 展示 experiment results
- 上傳影片到 YouTube / Google Drive
- 將影片連結放到 README

B 產出

```
docs/demo_script.md
demo video link
```

B-7. Final Submission Cleanup

B 主責，A 協助。

細項

- 檢查 `README.md`
- 檢查 `eval.txt`
- 檢查 `.gitignore`
- 確認沒有 push：
  - `.o`
  - executable binary
  - core dump
  - unnecessary logs
  - API key
- 確認 repo 可以從零安裝和執行
- 如果 repo private，要加入 instructor `ric2k1`
- 最後提交前檢查 demo video link 可開啟

26. A/B 分工總表

工作項目	A	B
RTL bug case design	主責	協助檢查
Buggy RTL	主責	不負責
Fixed RTL	主責	不負責
Testbench	主責	協助檢查 log 格式
Ground truth	主責	協助轉成 experiment format
Simulation runner	主責	協助整合
Failure log parser	主責	協助定義 LLM 欄位
RTL code extractor	主責	協助定義 prompt input
CLI main flow	主責	協助接 LLM API
Interactive mode	主責	協助設計 demo flow
Batch run script	主責	協助測試
Fix verification	主責	放入 report
LLM API integration	不負責	主責
Prompt design	提供 RTL context	主責
LLM output format	協助檢查 correctness	主責
Markdown debug report	提供 intermediate JSON	主責
Experiment comparison	提供 raw results	主責整理
README run/test section	協助撰寫	主責整合
README final report	提供 technical details	主責

工作項目	A	B
Demo script	提供 demo command	主責
Demo video	協助確認流程	主責
Final repo cleanup	協助	主責
`eval.txt`	共同確認	共同確認

27. Contribution 建議

如果兩人照此分工完成，建議：

```
MemberA    50
MemberB    50
```

理由：

- A 負責 benchmark、simulation、parser、extractor、CLI、fix verification，是 core implementation 主體
- B 負責 LLM integration、prompt、report、experiment、README、demo video，是 AI flow 與 final deliverables 主體
- 兩邊工作性質不同，但工作量與專案價值接近

28. Demo Video 腳本

影片上限 6 分鐘。

0:00 到 0:40：Motivation

說明 RTL debugging 通常需要同時看：

- RTL code
- simulation log
- waveform
- suspicious signals
- signal relation

這個流程很花時間，因此本專案設計 AIRTLDebug，用 simulation evidence 和 LLM 協助定位 bug root cause。

0:40 到 1:20：System Architecture

介紹 pipeline：

```
Case Loader
Simulation Runner
Failure Log Parser
RTL Context Extractor
Prompt Builder
LLM Debug Agent
Debug Report Generator
Fix Verification
```

1:20 到 2:40：Live Demo: Batch Debug Flow

展示一個 case：

```
python src/rtl_debug_agent.py \
--case benchmarks/counter_off_by_one \
--model gemini \
--fix-check
```

展示：

- simulation failed

- failure cycle
- failed signal
- expected / actual
- suspicious signals
- extracted RTL block

2:40 到 3:10：Interactive Debug Mode

展示 interactive mode：

```
python src/rtl_debug_agent.py \
  --case benchmarks/counter_off_by_one \
  --model gemini \
  --interactive
```

展示使用者可以選擇：

1. Show failure log
2. Show suspicious RTL block
3. Ask LLM for root cause
4. Run fixed RTL verification
5. Export debug report
0. Exit

這段用來說明 AIRTLDebug is not only a batch report generator. It also supports a simple user-guided debugging flow.

3:10 到 4:00：LLM Debug Report

展示 generated debug report：

- root cause
- evidence from simulation log
- suspicious RTL region
- suggested fix
- confidence score

4:00 到 4:40：Fix Verification

展示 fixed RTL simulation PASS。

說明：

- tool 不直接修改 RTL
- LLM 給 suggested fix
- prepared fixed RTL 用於 verification
- verification result 放進 report

4:40 到 5:30：Experimental Results

展示三種設定比較：

- LLM only
- LLM + log
- Full AIRTLDebug flow

展示 final result table。

5:30 到 6:00：Conclusion

總結：

- AIRTLDebug 建立一個 simulation-guided RTL debugging agentic flow
- 可以產生 structured debug report
- 可以定位 counter、FIFO、FSM bug 的 root cause
- 實驗比較 LLM only、LLM + log、full flow 的差異

29. 最小可交版本

如果時間不夠，至少完成：

1. 兩個 RTL bug cases
2. 每個 case 有：
    - buggy RTL
    - fixed RTL
    - testbench
    - spec
    - expected root cause
    - ground truth
3. CLI 可以跑 simulation
4. CLI 可以 parse failure log
5. CLI 可以擷取 suspicious RTL block
6. CLI 可以呼叫 LLM 產生 debug report
7. README 有完整 compile / run / test 說明
8. Demo video 展示至少一個完整 debug flow
9. `eval.txt` 完成
10. third-party disclosure 完成

這樣可以符合題目核心要求。

30. 加分版本

如果時間足夠，可以加入：

1. `--interactive`
    - 互動式查看 failure log、RTL block、LLM analysis、fix verification
2. `--no-llm`
    - 只跑 simulation、parser、RTL extraction
3. `--fix-check`
    - 自動跑 fixed RTL 確認 PASS
4. `--model mock`
    - 避免 demo 時 API 失敗
5. JSON output
    - 方便 experiment 統計
6. prompt comparison
    - 比較 LLM only、LLM + log、full flow
7. simple waveform summary
    - 從 VCD 或 testbench print 中整理 signal trace

8. `--emit-patch`
    - 讓 LLM 輸出 patch suggestion，但不直接覆蓋 source code

31. GitHub Workflow

每次開始工作前

```
git pull
```

A 新增 benchmark case

```
git checkout -b add-fifo-bug-case
git add benchmarks/fifo_full_flag_bug
git commit -m "Add FIFO full flag bug benchmark"
git push
```

A 新增 log parser

```
git checkout -b add-log-parser
git add src/log_parser.py
git commit -m "Add simulation log parser"
git push
```

B 新增 LLM client

```
git checkout -b add-llm-client
git add src/llm_client.py prompts/
git commit -m "Add LLM client and debug prompts"
git push
```

B 更新 README

```
git checkout -b update-readme-report
git add README.md docs/
git commit -m "Update final project report"
git push
```

建議分支

```
main
  add-counter-case
  add-fifo-case
  add-vending-machine-case
  add-sim-runner
  add-log-parser
  add-rtl-extractor
  add-llm-agent
  add-report-writer
  add-experiments
  update-readme
```

合併方式

建議用 Pull Request，不要兩個人同時直接推 main。

32. .gitignore 建議

```
# Python
__pycache__/
*.pyc
.venv/
venv/

# Environment
.env

# Simulation outputs
*.out
*.vcd
*.fst
*.o
core
core.*

# Generated outputs
outputs/*/
!outputs/.gitkeep

# OS files
.DS_Store
Thumbs.db
```

33. Final Submission Checklist

Repo

- [ ] `README.md`
- [ ] `eval.txt`
- [ ] `requirements.txt`
- [ ] `.env.example`
- [ ] `.gitignore`
- [ ] `src/`
- [ ] `benchmarks/`
- [ ] `prompts/`
- [ ] `scripts/`
- [ ] `docs/`
- [ ] `outputs/.gitkeep`
- [ ] `docs/sample_reports/`

Benchmark

- [ ] counter case buggy version FAIL
- [ ] counter case fixed version PASS
- [ ] FIFO case buggy version FAIL
- [ ] FIFO case fixed version PASS
- [ ] vending machine case buggy version FAIL
- [ ] vending machine case fixed version PASS
- [ ] each case has `ground_truth.json`

Tool

- [ ] CLI can run one case
- [ ] CLI can run all cases
- [ ] simulation runner works

- [ ] log parser works
- [ ] RTL Context extractor works
- [ ] prompt builder works
- [ ] LLM client works
- [ ] report writer works
- [ ] fix-check works
- [ ] no-llm mode works
- [ ] interactive mode works

Report

- [ ] group members
- [ ] compile / run / test instructions
- [ ] key contributions
- [ ] prior work / third-party disclosure
- [ ] algorithm pseudo code
- [ ] algorithm description
- [ ] implementation details
- [ ] benchmark cases
- [ ] experimental setup
- [ ] experimental results
- [ ] limitations
- [ ] references
- [ ] demo video link

Submission

- [ ] GitHub repo is ready
- [ ] add instructor `ric2k1` if repo is private
- [ ] no `.o` files
- [ ] no executable binary
- [ ] no core dump
- [ ] no unnecessary logs
- [ ] no API key
- [ ] final push before 2026/06/20 21:00